ADVANCED METHODS OF AI · WORKSHEET 3

Asking, Defining, Constraining, Deriving, Generating

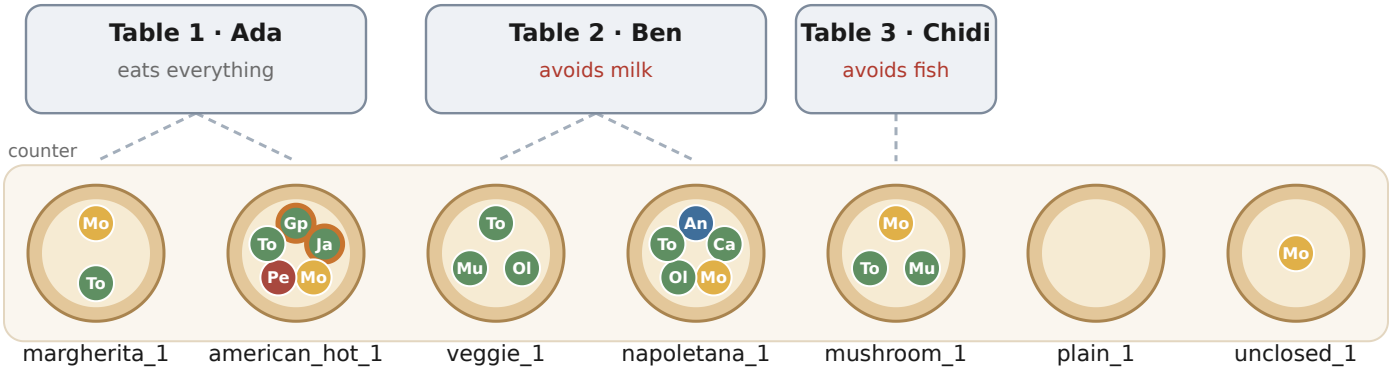
Worksheet 3 · querying a knowledge base with the Entity Query Language

> **Now:** in pairs, open `03-pizza_queries.ipynb` (link on Stud.IP) and run its **setup cell**. It installs EQL while we talk.

WHERE WE ARE

Last week: represent. Today: use.

- Worksheet 2: the pizza world in first-order logic, in OWL, and as Python classes.
- Today a service robot in a pizzeria *uses* that knowledge. Everything it does starts with one kind of expression: a **description**.



LOGIC

A query is a formula with free variables

$$\text{Pizza}(x) \land \exists t\, (\text{hasTopping}(x, t) \land \text{CheeseTopping}(t))$$

- The picture of the pizzeria is an **interpretation**: a domain of objects and the extension of every predicate.
- The **answers** are the values of the free variable x that make the formula true in it.
- That is *checking* one model, not proving that something holds in *every* model. The difference will matter: does the robot know everything?

EQL: the same formula, over Python objects

```
cheesy = a(Pizza)(toppings=a(CheeseTopping)()).from_(pizzas)
```

EQL	first-order logic
`a(Pizza)`	a variable x with Pizza(x)
`toppings=a(CheeseTopping)()`	$\exists t$ (hasTopping(x, t) $\wedge$ CheeseTopping(t)): *some* element of the list
`.from_(pizzas)`	the objects x ranges over; left out: the registry, every object the robot created
`cheesy.tolist()`	all values of x that satisfy the formula

Read it aloud: *“a pizza with a cheese topping, from the pizzas”*. Writing a query computes nothing; `.tolist()` evaluates it.

EQL

Beyond patterns: naming variables

```
pizza   = a(Pizza).from_(pizzas)
topping = flat_variable(pizza.toppings)
pizza.where(for_all(topping, HasType(topping, VegetableTopping)))
```

$$\text{Pizza}(x) \land \forall t\,(\text{hasTopping}(x, t) \rightarrow \text{VegetableTopping}(t))$$

- A pattern says *some*. For *every* and *no*, name the topping and use `for_all`, `not_(exists(…))`.
- Every `a(…)` is a variable of its own: two `a(Pizza)` are two pizzas.
- `set_of(x, y)` returns several free variables: the answers become assignments.

LOGIC

A class declaration is already a set of axioms

```
class MozzarellaTopping(CheeseTopping): ...
class Pizza:
    toppings: list[PizzaTopping]
class VegetarianPizza(Role[Pizza]): ...
```

KRROOD reads	first-order logic
a subclass	$\forall x\,(\text{MozzarellaTopping}(x) \rightarrow \text{CheeseTopping}(x))$
a typed attribute	$\forall x\,\forall t\,(\text{Pizza}(x) \wedge \text{hasTopping}(x, t) \rightarrow \text{PizzaTopping}(t))$
a role of a pizza	$\forall v\,(\text{VegetarianPizza}(v) \rightarrow \text{Pizza}(\text{roleTaker}(v)))$

All are *necessary* conditions: what follows from being a C. A **definition** adds the *sufficient* one, $\text{C}(x) \leftrightarrow \varphi(x)$, and only that lets the robot decide what is a C. (Act 2)

TODAY

One description, five uses

Act	Story	The description is …	Logic
1 Asking	“Which pizzas have cheese?” “*The* one with olives.”	a question	∃, ∀, ¬ in one interpretation
2 Defining	“Which pizzas may carry the green V?”	a definition	↔: check, classify, conclude
3 Constraining	“Ben must never get milk.”	a constraint	¬∀ ≡ ∃¬
4 Deriving	the kitchen writes allergen warnings	the body of a rule	∃ in the head: new objects
5 Generating	Dana orders a pizza that does not exist	a specification	constructing a model

The expression stays the same. Asking and constraining *read* what the robot knows; defining, deriving and generating *write* to it. A rule's new objects are forced, a generator's are chosen.

TODAY

Two questions to keep in mind

- **Does the robot know everything?** The query treats what it knows as complete; last week's OWL reasoner did not. Who is right about `unclosed_1`? (Act 1)
- **What happens when the world changes?** A cook makes a mistake at 19:50. Which of the robot's conclusions still hold? (Act 4)

TODAY

How we work today

- In pairs, one laptop. The **sheet** is for logic and predictions, the **notebook** for running EQL.
- **Predict, then run.** Write your answer on the sheet before you run a cell: a wrong prediction is where you learn the most.
- Every cell runs as it is; a `TODO` you cannot solve does not stop you.
- Every answer is drawn on the picture of the pizzeria. Draw along on Figure 2 of the sheet.

Slides 8′, then:

Prologue	Ask	Define	Constrain	Derive	Generate	Wrap
4′	24′	18′	12′	12′	7′	5′